

ShapeArea SOAP Web Service

Project Documentation

Student Name: Abenezer Tewodros

ID: 145267

Department: Software Engineering

Project Title: ShapeArea – SOAP Web Service for Calculating Areas of Geometric Shapes

1. Project Overview

This project implements a **SOAP-based web service** using **Spring Boot + Spring Web Services (Spring-WS)** that calculates the area of five geometric shapes:

- Circle
- Square
- Rectangle
- Parallelogram
- Triangle

The service follows **contract-first** design: the XSD schema is defined first, then the WSDL and Java endpoint are generated from it.

2. Design Decisions

- **Contract-First Approach:** XSD was created first to define strict data types and structures. This guarantees interoperability and validation.
- **Document/Literal Style:** Used for better performance and standard compliance.
- **PositiveDouble Custom Type:** Prevents negative or invalid values at schema level.
- **Separate Request/Response Elements:** Each shape has its own request and response elements for clarity and future extensibility.
- **Namespace:** <http://example.com/shapetool> – clear, versionable, and avoids conflicts.
- **Validation:** Enabled at both schema level (XSD restrictions) and runtime (PayloadValidatingInterceptor).
- **Error Handling:** Automatic SOAP Faults via Spring-WS validation + custom exceptions if needed.

3. XSD Design and Schema Structure

File: <src/main/resources/schemas/shapetool.xsd>

Key Design Features:

- **Target Namespace:** <http://example.com/shapetool>
- **Custom Simple Type:** [PositiveDouble](#) (restricts values ≥ 0.0)
- **Element-per-Operation:** Each shape has dedicated request and response elements
- **Strict Structure:** Uses `<xs:sequence>` for ordered parameters

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
```

```
targetNamespace="http://example.com/shapetool"
```

```

    xmlns:tns="http://example.com/shapetool"

    elementFormDefault="qualified">

<xs:simpleType name="PositiveDouble">

    <xs:restriction base="xs:double">

        <xs:minInclusive value="0.0"/>

    </xs:restriction>

</xs:simpleType>

<!-- Circle -->

<xs:element name="CircleAreaRequest">

    <xs:complexType>

        <xs:sequence>

            <xs:element name="radius" type="tns:PositiveDouble"/>

        </xs:sequence>

    </xs:complexType>

</xs:element>

<xs:element name="CircleAreaResponse">

    <xs:complexType>

        <xs:sequence>

            <xs:element name="area" type="xs:double"/>

        </xs:sequence>

    </xs:complexType>

</xs:element>

<!-- Square, Rectangle, Parallelogram, Triangle (similar structure) -->

<!-- ... (full XSD as provided in previous messages) -->

```

</xs:schema>

Why this design?

- Reusability of **PositiveDouble**
- Clear separation of concerns per shape
- Easy to extend (add new shapes later)

4. WSDL and Service Operations

WSDL URL (after running the application): **http://localhost:8080/Shapearea/Shapearea.wsdl**

Service Name: **ShapeareaService**

PortType: **ShapeareaPort**

Binding: Document/Literal over HTTP

Operations (5 total):

Operation	Input Element	Output Element	Description
CalculateCircleArea	CircleAreaRequest	CircleAreaResponse	Area = $\pi \times r^2$
CalculateSquareArea	SquareAreaRequest	SquareAreaResponse	Area = side ²
CalculateRectangleArea	RectangleAreaRequest	RectangleAreaResponse	Area = length $\times$ width
CalculateParallelogramArea	ParallelogramAreaRequest	ParallelogramAreaResponse	Area = base $\times$ height
CalculateTriangleArea	TriangleAreaRequest	TriangleAreaResponse	Area = $0.5 \times$ base $\times$ height

The WSDL was auto-generated by **DefaultWsd11Definition** bean.

5. Service Implementation (Endpoint)

Class: **ShapeToolEndpoint.java**

@Endpoint

```
public class ShapeToolEndpoint {
```

```
    private static final String NAMESPACE_URI = "http://example.com/shapetool";
```

```
    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "CircleAreaRequest")
```

```
    @ResponsePayload
```

```
    public CircleAreaResponse circleArea(@RequestPayload CircleAreaRequest request) {
```

```

double area = Math.PI * request.getRadius() * request.getRadius();

CircleAreaResponse response = new CircleAreaResponse();

response.setArea(area);

return response;

}

// Similar methods for Square, Rectangle, Parallelogram, Triangle...

}

```

6. Spring-WS Configuration (**WebServiceConfig.java**)

Key beans:

- **MessageDispatcherServlet** mapped to **/Shapearea/***
- **DefaultWsd11Definition** named **Shapearea**
- **SimpleXsdSchema** loading **shapetool.xsd**
- **PayloadValidatingInterceptor** (request + response validation)

7. Validation Strategy

- **Schema-level:** **PositiveDouble** restriction + XSD structure
- **Runtime:** **PayloadValidatingInterceptor** with **validateRequest=true** and **validateResponse=true**
- Any invalid input (negative value, missing element, wrong namespace) automatically returns a SOAP Fault before reaching the endpoint method.

8. Error Handling / SOAP Fault Handling

- **Automatic Validation Faults:** Spring-WS throws **SoapFault** with detailed messages when XSD rules are violated.
- **Example Fault** (negative radius):

```

<soap:Fault>

  <faultcode>soap:Client</faultcode>

  <faultstring>Validation error: Value '-5.0' is not facet-valid with respect to minInclusive '0.0'</faultstring>

</soap:Fault>

```

Custom exceptions can be added later using `@SoapFault` annotation if needed.

9. Running the SOAP Web Service

1. Run the Spring Boot application.
2. Open browser and go to: <http://localhost:8080/Shapearea/Shapearea.wsdl> (You should see the full WSDL XML)

Service Endpoint: <http://localhost:8080/Shapearea>

10. Conclusion

This project successfully demonstrates a complete **contract-first SOAP web service** using Spring-WS with proper XSD design, validation, and error handling. The service is production-ready, easily extensible, and strictly follows SOAP/WSDL standards.

All required features (validation, namespace management, fault handling) are fully implemented and tested.

Project Files Summary:

- shapetool.xsd
- WebServiceConfig.java
- ShapeToolEndpoint.java
- Shapearea.wsdl (auto-generated)